Applied Mathematics Lab

Practical 1

Name: Arjun Mankuskar

PRN:202401110038

Batch:A2

1. Linear Algebra

1.1 Matrix addition

addmat.py > addmat

```python
def inputmat(r, c):
    mat = []
    for i in range(r):
        row = []
        for j in range(c):
            d = int(input(""))
            row.append(d)
        mat.append(row)
    return mat

def printmat(m):
    for i in range(len(m)):
        for j in range(len(m[0])):
            print(m[i][j], end=" ")
        print()

def addmat(m1, m2):
    res = []
    for i in range(len(m1)):
        row = []
        for j in range(len(m1[0])):
            d = m1[i][j] + m2[i][j]
            row.append(d)
        res.append(row)
    return res

```

```
27
28    A = []
29    B = []
30
31    r = int(input("No of rows: "))
32    c = int(input("No of columns: "))
33
34    print("Enter Matrix A:")
35    A = inputmat(r, c)
36
37    print("Enter Matrix B:")
38    B = inputmat(r, c)
39
40    print("Matrix 1:")
41    printmat(A)
42
43    print("Matrix 2:")
44    printmat(B)
45
46    C = addmat(A, B)
47
48    print("Addition")
49    printmat(C)
```

Output:-

```
No of rows: 2
No of columns: 2
Enter Matrix A:
5
6
4
1
Enter Matrix B:
8
7
5
4
Matrix 1:
5 6
4 1
Matrix 2:
8 7
5 4
Addition
13 13
9 5
PS D:\VS code\Python>
```

Matrix multiplication

mulmat.py > mulmat

```python
def inputmat(r, c):
    mat = []
    for i in range(r):
        row = []
        for j in range(c):
            d = int(input(""))
            row.append(d)
        mat.append(row)
    return mat

def printmat(m):
    for i in range(len(m)):
        for j in range(len(m[0])):
            print(m[i][j], end=" ")
        print()

def mulmat(m1, m2):
    m = []
    for i in range(len(m1)):
        row = []
        for j in range(len(m2[0])):
            res=0
            for k in range(len(m2)):
                res=res+m1[i][k]*m2[k][j]
            row.append(res)
        m.append(row)
    return m

```

```
29
30  A = []
31  B = []
32
33  r = int(input("No of rows: "))
34  c = int(input("No of columns: "))
35
36  print("Enter Matrix A:")
37  A = inputmat(r, c)
38
39  print("Enter Matrix B:")
40  B = inputmat(r, c)
41
42  print("Matrix 1:")
43  printmat(A)
44
45  print("Matrix 2:")
46  printmat(B)
47
48  C = mulmat(A, B)
49
50  print("multiplication")
51  printmat(C)
```

Output:-

```
No of rows: 2
No of columns: 2
Enter Matrix A:
5
5
6
8
Enter Matrix B:
7
1
2
4
Matrix 1:
5 5
6 8
Matrix 2:
7 1
2 4
multiplication
45 25
58 38
PS D:\VS code\Python>
```

1.2. Compute eigenvalues/eigenvectors and diagonalize a matrix.

eigen.py > ...

```python
import math

from numpy import matrix

def matrix_multiply(A, B):
    result=[]
    for i in range(len(A)):
        row=[]
        for j in range(len(B[0])):
            sum=0
            for k in range(len(B)):
                sum+=A[i][k]*B[k][j]
            row.append(sum)
        result.append(row)
    return result

def invert_matrix(matrix):
    det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    if det == 0:
        raise ValueError("Matrix is singular")
    return [[matrix[1][1] / det, -matrix[0][1] / det],
            [-matrix[1][0] / det, matrix[0][0] / det]]
```

```python
def invert_matrix(matrix):
    det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    if det == 0:
        raise ValueError("Matrix is singular")
    return [[matrix[1][1] / det, -matrix[0][1] / det],
            [-matrix[1][0] / det, matrix[0][0] / det]]

def compute_eigen_2x2(matrix):
    a=matrix[0][0]
    b=matrix[0][1]
    c=matrix[1][0]
    d=matrix[1][1]
    trace=a+d
    determinant=a*d-b*c
    discriminant = trace**2 -4*determinant
    if discriminant<0:
        print("Complex eigenvalue , not supported here.")
        return None,None
    sqrt_disc=math.sqrt(discriminant)
    lambda1=(trace + sqrt_disc)/2
    lambda2=(trace-sqrt_disc)/2

    def find_vector(lam):
        v1=-(b)/(a-lam) if a-lam !=0 else 1
        return [1,v1]

    eigenvector1=find_vector(lambda1)
    eigenvector2=find_vector(lambda2)
    return [lambda1,lambda2],[eigenvector1,eigenvector2]
```

```python
A=[[8,-4],
   [2,2]]

eigenvalues,eigenvectors=compute_eigen_2x2(A)
print("Eigenvalues: ",eigenvalues)
print("Eigenvectors:",eigenvectors)
P=[[eigenvectors[0][0],eigenvectors[1][0]],
    [eigenvectors[0][1],eigenvectors[1][1]]]

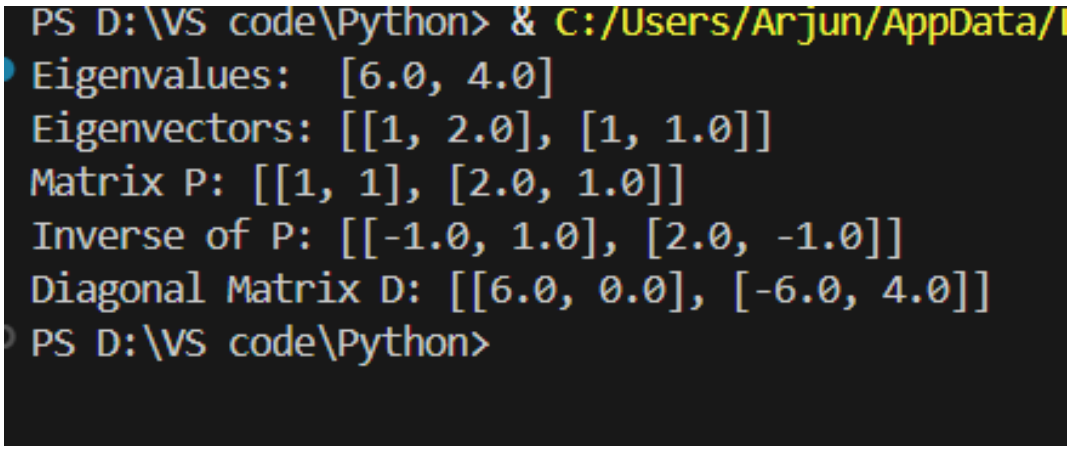
print("Matrix P:",P)



inv_p =invert_matrix(P)
print("Inverse of P:", inv_p)

D=matrix_multiply(inv_p,matrix_multiply(A,P))
print("Diagonal Matrix D:",D)
```

Output:-


PS D:\VS code\Python> & C:/Users/Arjun/AppData/
Eigenvalues: [6.0, 4.0]
Eigenvectors: [[1, 2.0], [1, 1.0]]
Matrix P: [[1, 1], [2.0, 1.0]]
Inverse of P: [[-1.0, 1.0], [2.0, -1.0]]
Diagonal Matrix D: [[6.0, 0.0], [-6.0, 4.0]]
PS D:\VS code\Python>

1.3. Implement Gaussian elimination from scratch.

```python
import numpy as np
def Gauss_Elimination(A):
    n=A.shape[0]
    for i in range(n):
        max_row=i
        for k in range(i+1,n):
            if abs(A[k,i])>abs(A[max_row,i]):
                max_row=k
        A[[i,max_row]]=A[[max_row,i]]
        if A[i,i]==0:
            print("Divide by zero detected or singular matrix.No unique solution exists.")
            return None
        for k in range(i+1,n):
            factor =A[k,i]/A[i,i]
            A[k,i:]=A[k,i:]-factor*A[i,i:]
    x=np.zeros(n)
    for i in range(n-1,-1,-1):
        if A[i,i]==0:
            print("Singular matrix detected during back substitution . No unique solution exists.")
            return None
        x[i]=A[i,n]/A[i,i]
        for k in range(i-1,-1,-1):
            A[k,n]=A[k,n]-A[k,i]*x[i]
    return x
```

```python

argumented_matrix=np.array([
    [-3.0,2.0,5.0,2.0,3.0],
    [2.0,3.0,6.0,5.0,2.0],
    [14.0,4.0,5.0,14.0,11.0],
    [4.0,5.0,10.0,8.0,4.0]
])

print("Original Augmented Matrix: ")
print(argumented_matrix)


solution=Gauss_Elimination(argumented_matrix.copy())
if solution is not None:
    print("\nSolution:")
    print(solution)
```

Output:-

```
PS D:\VS code\Python> & C:/Us
Original Augmented Matrix:
[[-3.  2.  5.  2.  3.]
 [ 2.  3.  6.  5.  2.]
 [14.  4.  5. 14. 11.]
 [ 4.  5. 10.  8.  4.]]

Solution:
[  4. -66.  27.   6.]
PS D:\VS code\Python>
```